

The Program Arrives Late

Theory Lag and Programming Before Understanding

Watson Hartsoe

Final Working Paper - August 2026

Abstract

Natural-language programming is usually described as a change in notation: instead of writing formal code, a programmer states an intention in ordinary language and a generative model translates that intention into executable form. This account places too much explanatory weight on the prompt and assumes too much coherence in the intention that precedes it. Programming from incomplete specifications long predates large language models, and designers routinely act before problems are fully determined. The more consequential change introduced by generative systems is that increasingly heterogeneous forms of unresolvedness can remain computationally productive. Procedures, categories, examples, aesthetic criteria, contextual references, and even the meaning of the desired transformation can remain unsettled while substantial artifacts are already being generated, inspected, rejected, and revised.

I call the resulting condition **theory lag**: the interval in which a human-computational system can already produce and discriminate among operative realizations while the human's explicit theory remains insufficient to explain those discriminations or guide unforeseen modification. Theory lag reveals three mechanisms that often appear identical at the interface: **task location**, in which an expression elicits behavior already latent in a model; **specification completion**, in which provisional realizations expose or supply commitments missing from the initial description; and **tacit steering**, in which human judgment guides generation without becoming fully articulable as specification. These mechanisms shift the unit of programming away from the isolated prompt toward a recursive relation among description, situated interpretation, realization, judgment, and revision.

Following Peter Naur's account of programming as theory building, I argue that successful generation is an inadequate test of whether a program's theory has been preserved. The stronger test is **modification competence**: whether a representation can support principled change when circumstances depart from those under which it was produced. Natural-language programming therefore raises a question more difficult than whether prompts count as code: **what must be preserved when execution can begin before understanding is complete?**

Keywords: natural-language programming; generative AI; specification; program synthesis; prompting; situated action; tacit knowledge; theory building

1. Before the Requirement Exists

Imagine telling a generative system:

Make the house remember everyone who has lived here.

A house appears whose walls display photographs of former residents.

No.

The house should remember them. It should not display records about them.

The next version alters its lighting according to patterns associated with former occupants.

Closer.

Memory should change what the house does, but nobody living there should be able to inspect a complete archive.

A third house appears.

The common description of this exchange is straightforward. The first prompt was underspecified. The model produced an incorrect implementation. Subsequent prompts added missing requirements. Through iteration, the user approached a sufficiently complete specification.

But this description assumes that the missing requirement already existed.

Perhaps before seeing the photograph-covered walls, the user had never distinguished **memory as archive** from **memory as altered behavior**. The first realization does not merely violate an existing specification. It makes a distinction available that the original description did not contain.

The distinction can then become part of the next instruction.

Something has therefore happened between intention and specification.

The artifact has participated in the production of the requirement by which the artifact is judged.

This is where natural-language programming becomes difficult to describe using the conventional sequence:

requirement -> program -> execution

In generative interaction, the relation can become recursive:

provisional description -> realization -> judgment -> revised
description

and, more importantly,

realization -> newly available distinction -> reconstituted requirement

The result is not simply a faster route from intention to implementation. The route can alter the intention itself.

2. Incompleteness Is Not New

It would be easy to overstate what has changed.

Programming did not previously require every detail of a program to be settled before computation began. Program synthesis has long accepted incomplete specifications. Programming-by-example can infer procedures from demonstrations rather than explicit algorithms. Interactive program synthesis makes incompleteness central to the programming process, using incremental computation and feedback to refine what users want (Le et al. 2017). Designers likewise do not wait until a problem is fully articulated before acting. Sketches, prototypes, mock-ups, and provisional arrangements regularly disclose properties of a problem that could not be derived from reflection alone. Recent empirical work in visualization design likewise describes expert practitioners making unresolved situations actionable through provisional local moves (Parsons et al. 2026).

The historical claim, then, cannot be:

Generative AI makes it possible to begin before the specification is complete.

That happened already.

A better question is:

What can remain unresolved while useful computation continues?

Earlier systems commonly restrict unresolvedness through an established representational regime. A synthesis system may leave the procedure unknown while fixing the domain-specific language in which the solution must be expressed. A compiler may postpone values until runtime while preserving a highly explicit operational semantics. A programming-by-example system may infer transformations while constraining the kinds of examples it can interpret.

Generative models tolerate a more heterogeneous field of missingness.

The unresolved part can be procedural:

Find a better way to organize these records.

It can be aesthetic:

This feels too corporate.

Deictic:

Keep this part.

Analogical:

Make it behave more like a garden than a filing cabinet.

Narrative:

The house should remember, but it should never confess.

Evaluative:

Again. The last one understood the concept too literally.

These are not merely unspecified parameter values. They are descriptions whose operative significance depends on contextual interpretation, learned regularities, examples, prior artifacts, and subsequent judgment.

The transformation is therefore better described as an expansion in the **kinds of unresolvedness that can remain computationally viable**.

The user is permitted to begin before many distinctions have been translated into the vocabulary from which conventional specifications are made.

3. Theory Lag

Peter Naur's "Programming as Theory Building" provides a useful way to understand why this matters.

For Naur, the central achievement of programming is not the program text itself. The programmer develops a theory connecting the program to the portion of the world it is intended to handle. Possessing that theory means more than knowing what the existing program does. It includes knowing why its structures are present, how they correspond to relevant affairs, and how they ought to change when new demands arise (Naur 1985).

This is why maintenance is so important to Naur's account. Program text and documentation can survive while the understanding needed to modify the program intelligently disappears.

Generative programming introduces a peculiar temporal possibility into this picture.

A functioning realization may appear **before** the human possesses anything like the theory Naur describes.

The human can run it.

Reject it.

Notice a difference.

Retain one behavior.

Discard another.

Change the request because the generated artifact reveals that the original request was badly framed.

Substantial computation can proceed while explicit understanding remains incomplete.

I call this **theory lag**.

Theory lag is the interval in which a human-computational system can already produce, evaluate, and revise operative realizations while the human's explicit theory remains insufficient to account for the distinctions guiding those revisions or to determine how the system should respond to unforeseen change.

Theory lag should not be confused with ignorance. A practitioner inside the lag may possess enormous expertise. They may know immediately that something is wrong. They may know which candidate is stronger. They may recognize a violation before they can state the violated rule.

The important asymmetry is simply this:

capacity to discriminate > capacity to explicate

Generative systems make this asymmetry computationally productive.

The next substantial realization does not have to wait for the gap to close.

4. When the Artifact Produces the Requirement

Theory lag matters most where realization does not simply test a requirement but helps make that requirement interpretable.

Suppose the user asks:

Make the interface calm.

The system produces a pale interface with generous white space.

The user rejects it:

Not spa calm. Calm because nothing is competing for attention.

That correction contains a distinction.

But where was the distinction before the failed interface?

One possibility is that the user already possessed it and merely failed to articulate it. The generation functions as requirements elicitation.

Another possibility is more interesting. The contrast between *spa calm* and *noncompetitive calm* becomes available only because a concrete realization has separated them.

The failed artifact has performed an epistemic operation.

It has given the practitioner something sufficiently definite to negate.

This yields a different account of iteration:

$$D_t \rightarrow Y_t \rightarrow J_t \rightarrow D_{(t+1)}$$

where D_t is not a fixed requirement receiving additional detail. It is a description whose meaning may itself be transformed by the judgment applied to Y_t .

The next description can therefore contain distinctions that did not exist propositionally in the previous one.

That is why *feedback* is too weak a word.

Feedback suggests that a known target exists and that error tells us how far the current realization lies from it.

Generative iteration can instead change the target space.

The user is not merely moving toward an object.

The succession of objects can change what *toward* means.

5. Three Different Things Called Prompting

The interface makes this process look simpler than it is.

A person types a small amount of language. A system produces a much larger structure. It is tempting to explain every such event by saying that the model "filled in the gaps."

At least three different mechanisms can produce this appearance.

5.1 Task location

Reynolds and McDonell propose that some prompting behavior is better understood as **task location**. Rather than teaching the model a new task at inference time, an expression can help locate behavior already learned during training (Reynolds and McDonell 2021).

Under this account:

$$\text{expression} \rightarrow \text{latent behavioral region} \rightarrow \text{realization}$$

A sparse prompt can produce elaborate behavior because the prompt is sparse while the larger operative system is not.

"Translate this as a meticulous philologist" may work not because the sentence specifies philological translation but because it selects among capabilities already available within the model.

5.2 Specification completion

Other cases require stronger interpretive commitment.

"Make the house remember" does not merely select one obvious task. The realization must settle dimensions the user has not determined. Memory becomes behavioral rather than archival; spatial rather than propositional; persistent rather than episodic.

A generation therefore externalizes one possible completion of an incomplete description.

That completion can then expose a missing criterion. Empirical research on prompt programming likewise reports that developers sometimes discover requirements through interacting with model behavior and struggle to anticipate which assumptions must be made explicit (Liang et al. 2024).

partial description -> interpretive commitment -> realization ->
criterion discovery

5.3 Tacit steering

But even specification completion assumes that the process tends toward an explicit specification.

It may not.

Michael Polanyi's account of tacit knowing begins from the fact that human competence routinely exceeds what can be stated propositionally. One can identify a familiar face without possessing an explicit procedure for recognizing it (Polanyi 2009 [1966]).

Generative interaction creates an analogous possibility.

A writer can repeatedly reject sentences.

A designer can select one image from eight.

A programmer can say:

This architecture is closer, but the state model is still wrong.

The judgment can be stable and consequential while its underlying criterion remains only partially articulable.

The process may therefore continue as:

candidate -> discrimination -> adapted candidate set -> discrimination
-> ...

without terminating in a complete symbolic representation of the discriminating rule.

Call this **tacit steering**.

These mechanisms can coexist. But they locate the operative structure differently.

Task location places much of it in the learned model.

Specification completion distributes it across description and realization.

Tacit steering leaves an irreducible portion in ongoing human judgment.

A theory of prompting that calls all three "natural-language specification" loses precisely what needs explanation.

6. The Prompt Is Not the Unit

Consider the prompt:

Make this darker.

As autonomous text, it specifies almost nothing.

This may refer to an image, a paragraph, a selected region, a previously generated object, or an element inside an interactive environment. *Darker* may concern luminance, color, atmosphere, rhetoric, or narrative consequence.

Yet within the right situation, the expression may be entirely adequate.

Lucy Suchman's critique of plans helps explain why. A plan can function as a resource for situated action without determining the detailed course of that action. The significance of a representation depends partly upon the circumstances in which it is taken up (Suchman 1987).

This does not mean a language model is situated in the same sense as a human actor. It means that **precedence should not be confused with determination**. An expression occurring before an operation need not contain the operation in compressed form.

For generative systems, an operative event depends on something closer to:

$$E_t = (p_t, c_t, m, a_t, y_{(t-1)}, j_{(t-1)})$$

where:

- `p\_t` is the visible expression;
- `c\_t` is the addressable context;
- `m` is the interpreting model;
- `a\_t` is the available repertoire of operations;
- `y\_{(t-1)}` is the previous realization;
- `j\_{(t-1)}` is prior human judgment.

Change one of these while leaving the words untouched and the operative meaning may change.

The prompt is therefore not a self-sufficient program.

It is a **move within a changing coupled state**.

This explains the curious power of expressions such as:

This.

Again.

Like the last one, except...

Keep the left half.

Their effectiveness comes precisely from information that is not inside them.

7. Description Does Not Become Operation

This also weakens a seductive description of generative systems:

Description becomes operation.

A description does not acquire an intrinsic operational essence.

Consider:

Delete this.

In a novel, nothing happens.

In an ordinary chat, the recipient may ask what *this* refers to.

In an image editor, a selected object may disappear.

In an agentic environment, a tool invocation may be produced.

In another environment, the operation may be blocked because the user lacks permission.

The words are constant.

The operative arrangement changes.

J. L. Austin's account of performative utterances is useful here because the familiar slogan "words do things" is only half of his argument. Performative force depends upon circumstances, procedures, roles, and conditions under which an act succeeds or misfires (Austin 1975).

Computational systems have analogous conditions.

An expression becomes consequential only when relevant conditions hold:

$F = \{\text{resolvable referent, recognized operation, authorized actor, available capability, admissible state, successful transition}\}$

The important phenomenon is therefore not executable description in isolation.

It is **interpretive coupling**: arrangements in which an interpreted representation can modify what states become reachable next.

This formulation returns the machinery that "description becomes operation" makes disappear.

It also exposes power.

Who defines what can be referenced?

Who chooses the interpreter?

Who grants tool access?

Which state transitions are available?

Which utterances are treated as instructions?

The apparent power of language is partly the power of the infrastructure to which language has been connected.

8. The Program Arrives Late

Return to the remembering house.

After enough iterations, the user may eventually be able to write something much more explicit:

- memory changes behavior rather than displaying records;
- traces of past occupants alter present affordances;
- memory is partial rather than total;
- occupants cannot inspect it directly;
- forgetting remains possible;
- remembrance should be sensed indirectly.

This object now looks recognizably like a specification.

But it did not exist at the beginning.

It is partly a retrospective stabilization of distinctions produced during interaction.

The theory arrived after versions of the system had already been generated.

Hence the central temporal reversal:

The program, understood as an explicit account of what must remain true, can arrive after the system has begun to run.

This does not mean there was no prior intention.

Nor does it mean the model authored the theory.

It means that the relation between theory and realization can become reciprocal.

At iteration t :

$T_t \rightarrow D_t \rightarrow Y_t \rightarrow J_t \rightarrow T_{(t+1)}$

$T_{(t+1)}$ need not simply contain more details than T_t

A generated realization may reveal that the relevant distinction was wrong, that two requirements were incompatible, or that the object under construction has become something different from the one initially imagined.

The theory can change kind.

9. Why Regeneration Proves Too Little

Once a sufficiently explicit theory has emerged, another temptation appears.

Perhaps it can be compressed.

A long program, essay, world, or dissertation could be represented by a smaller generative object capable of reproducing recognizable realizations.

If an 80,000-word argument can be regenerated from a compact prompt system, perhaps the prompt system is the deeper intellectual artifact.

Naur gives us a reason to resist this conclusion.

A representation can reproduce an artifact without preserving the theory required to alter it intelligently.

Let $R(P)$ denote the fidelity with which representation P regenerates previously observed realizations.

Now define $M(P, \Delta W)$ as the competence with which P , together with its interpreter, supports modification under an unforeseen change in relevant world conditions ΔW .

Nothing guarantees:

$R(P) \text{ increases} \rightarrow M(P, \Delta W) \text{ increases}$

A prompt can reproduce an essay because a model recognizes the genre and argument pattern.

It can reproduce software because common architecture is supplied by learned priors.

It can reconstruct an interface because examples constrain imitation.

None of these establishes that the representation preserves why the existing structures matter.

The harder test is **modification competence**.

Change the world.

Remove a premise.

Introduce evidence the argument cannot absorb.

Require the remembering house to forget.

Change the jurisdiction governing the software.

Move the interface from a single user to collaborative ownership.

Now ask:

Which structures should change?

Which should remain?

Why?

A representation preserving the relevant program theory should support coherent modification under such counterfactual pressure.

Regeneration asks whether the old world can be rebuilt.

Modification asks whether the representation knows how to survive a new one.

10. Making Theory Lag Falsifiable

Theory lag should not survive merely because it is an attractive description.

It suggests at least three empirical tests.

Criterion emergence

Before generation, elicit every criterion a participant can state.

After each realization, record new criteria.

Distinguish criteria that were:

- consciously known but omitted;
- recognized as relevant only after a violation;
- created by an unexpected possibility;
- merely post-hoc explanations.

If generations regularly precede stable articulation of consequential criteria, theory lag gains empirical support.

Judgment before explanation

Present alternatives and require immediate ranking or rejection before asking participants to explain their judgment.

Then compare consistency of discrimination with consistency and predictive power of explanation.

If people reliably discriminate among candidates while their explicit rules remain unstable or incomplete, operative judgment is outrunning explicit theory.

Regeneration versus modification

Construct a compact generative representation that reliably reproduces an existing artifact.

Then introduce unforeseen requirements.

Measure separately:

- regeneration fidelity;
- explanation of existing structures;
- identification of relevant invariants;

- quality of modification;
- preservation of original rationale.

If regeneration succeeds while principled modification collapses, realization has survived without sufficient theory.

These tests could also kill the concept.

If criteria are generally explicit before generation, theory lag adds little beyond ordinary requirements elicitation.

If post-generation explanations are mostly unstable rationalizations, realization may be producing preference drift rather than theory.

If modification competence closely tracks regeneration fidelity, the distinction between reproducing a system and preserving its theory may be weaker than proposed.

And if theory lag proves no greater in generative practice than in ordinary skilled design, its significance will need to be narrowed further.

11. The Economy of Provisional Worlds

If incomplete specification, situated action, iteration, and tacit judgment all predate generative AI, what remains distinctive?

Perhaps not a new logical structure.

Perhaps a new **economy of provisional realization**.

A sketch takes time.

A prototype takes more.

A functioning interface, substantial essay, architectural visualization, or alternative codebase traditionally requires increasing quantities of labor before it can be judged.

Generative systems compress that cost.

An unresolved description can produce a substantial candidate quickly enough that candidate-making itself becomes a routine mode of thought.

The number and heterogeneity of realizations available before a practitioner must commit can rise dramatically.

Call this **iteration bandwidth**.

High iteration bandwidth can improve inquiry.

More realizations expose more differences.

Previously invisible criteria become visible.

Unexpected possibilities alter the problem.

But the same abundance produces the opposite danger.

Every difficult passage now has another version.

Every unresolved interface has another arrangement.

Every architectural contradiction has another plausible image.

The machine may allow the practitioner to route around a difficulty before the difficulty becomes intellectually expensive enough to understand.

The danger is therefore not simply automation.

It is **evasion through abundance**.

Slow work sometimes forces theory because the cost of continuing without understanding becomes intolerable.

A system capable of generating indefinitely plausible next moves can keep theory lag open.

12. Designing for the Gap

This suggests a different agenda for natural-language programming environments.

Most systems are optimized to reduce friction between intention and result.

Better inference.

Shorter prompts.

More automatic context.

Fewer clarifying questions.

Greater ability to infer what the user "really meant."

But if theory lag is real, eliminating every friction can conceal the very places where theory remains inadequate.

A programming environment built around theory lag might therefore do something unusual.

It might preserve repeated corrections and ask whether they imply a new invariant.

It might distinguish a local patch from a change in the governing problem.

It might retain rejected realizations because they reveal boundaries of the emerging theory.

It might show which behaviors come primarily from the prompt and which appear to be supplied by model priors.

It might periodically introduce controlled perturbations:

What if this assumption disappears?

What if the data model changes?

What if there are three owners rather than one?

What if memory must decay?

The point would not be to force every intuition into a formal specification.

Some judgment may remain tacit.

The point would be to expose the difference between:

a system that continues to produce acceptable outputs

and

a practitioner who increasingly understands what must remain true.

These are not the same accomplishment.

13. Programming as Theory Building After Execution

Naur placed theory building at the center of programming because program text alone could not preserve the understanding required for intelligent change.

Generative systems do not make that theory unnecessary.

They alter when it may be acquired.

The practitioner can now encounter substantial realizations while important parts of the theory remain unsettled.

They can discover criteria through failure.

Locate behaviors through sparse expressions.

Steer by judgments they cannot fully articulate.

Allow previously informal descriptions to participate in consequential state changes.

Programming therefore becomes neither pure specification nor pure selection.

It becomes a process in which **theory and realization can develop together**.

That is the stronger interpretation of the familiar loop:

describe -> generate -> inspect -> correct -> regenerate

The loop is not important because iteration improves prompts.

It is important because each traversal can change the theory under which the artifact is judged.

The real output of the loop is therefore not only the latest artifact.

It is also the evolving account of what the artifact is supposed to be.

Conclusion

Natural-language programming should not be understood primarily as programming in English.

Nor does its significance lie in proving that prompts are a new species of code.

Programming from incomplete specifications already existed. Skilled action already exceeded explicit rules. Designers already learned from prototypes. Plans already depended on situations. Language already acquired consequences inside institutional arrangements.

Generative systems reorganize these older relations by making **heterogeneous unresolvedness cheaply operable**.

A programmer can now submit an intention before its consequential distinctions have been completely articulated, encounter substantial realizations, recognize differences that were previously unavailable, and feed those judgments back into the process without first translating them into a formal language.

This produces theory lag: execution and evaluation can proceed ahead of explicit understanding.

Inside that lag, the apparent simplicity of prompting conceals different mechanisms. Sometimes a small expression locates a capability already present in the model. Sometimes a realization supplies commitments that help constitute a specification. Sometimes the human steers through judgments that never become completely explicit.

None of these makes theory obsolete.

They make its delayed arrival possible.

And delayed theory creates a problem that successful generation cannot solve.

A representation may regenerate an artifact without preserving the understanding required to modify it coherently when circumstances change. The stronger test is therefore not reproduction but modification competence.

The old programming question was:

How precisely must the program be specified before it can run?

Generative systems make another question unavoidable:

How far can a program run before its programmer understands what must remain true?

That is not merely a question about prompts.

It is a question about the temporal order of programming itself.

When realization can precede understanding, the central task becomes preserving enough friction, history, judgment, and theory that understanding can still catch up.

The program can arrive late.

It cannot be permitted never to arrive.

References

- Austin, J. L. 1975. *How to Do Things with Words*. 2nd ed. Edited by J. O. Urmson and Marina Sbisa. Cambridge, MA: Harvard University Press. First published 1962.
- Le, Vu, Daniel Perelman, Oleksandr Polozov, Mohammad Raza, Abhishek Udapa, and Sumit Gulwani. 2017. "Interactive Program Synthesis." arXiv:1703.03539.
- Liang, Jenny T., Melissa Lin, Nikitha Rao, and Brad A. Myers. 2024. "Prompts Are Programs Too! Understanding How Developers Build Software Containing Prompts." arXiv:2409.12447.
- Naur, Peter. 1985. "Programming as Theory Building." *Microprocessing and Microprogramming* 15 (5): 253-261. [https://doi.org/10.1016/0165-6074\(85\)90032-8](https://doi.org/10.1016/0165-6074(85)90032-8).
- Parsons, Paul C., Prakash Shukla, Phuong Bui, Srishti Agrawal, and Ali Baigelenov. 2026. "Situatdness in Visualization Design: Making Unresolved Work Actionable." arXiv:2608.08274.
- Polanyi, Michael. 2009 [1966]. *The Tacit Dimension*. Chicago: University of Chicago Press.
- Reynolds, Laria, and Kyle McDonell. 2021. "Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm." arXiv:2102.07350.
- Suchman, Lucy A. 1987. *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge: Cambridge University Press.